

Confirm that calling `calculateMonthlySalary()` on a `Player` reference that actually holds a `StarPlayer` object will dispatch to `StarPlayer`'s overridden method, not `Player`'s — is this guaranteed by Java or could it silently call the wrong one?

Verified Java's guaranteed dynamic dispatch mechanism via JVM specification >

Yes — confirmed, and it's not just "usually works," it's a **language guarantee**, not a convention or a lucky coincidence.

Why it's guaranteed:

Java specifies that calls to non-static, non-private, non-final instance methods are resolved via **dynamic (virtual) dispatch**. The compiler emits an `invokevirtual` bytecode instruction for `player.calculateMonthlySalary()`, and that instruction explicitly tells the JVM: *look up the method using the object's actual runtime class, not the reference's declared/compile-time type*.

In your case:

```
java
Player player = playerList.findPlayerById(id); // declared type: Player
player.calculateMonthlySalary();               // dispatch uses runtime type
```

At the bytecode level, the JVM doesn't even consider that `player` is declared as `Player` for this decision — it walks the actual object's class (`StarPlayer`), finds its `calculateMonthlySalary()`, and calls that. This is part of the JVM specification itself, not something that depends on JIT optimization, garbage collector behavior, or anything non-deterministic.

Why it can't "silently call the wrong one" in y ↓ code specifically:

There's an extra safety net here beyond dispatch itself: `Player.calculateMonthlySalary()` is

Why it can't "silently call the wrong one" in your code specifically:

There's an extra safety net here beyond dispatch itself: `Player.calculateMonthlySalary()` is declared `abstract`. That means:

- There is no `Player` implementation to *accidentally* fall back to — it doesn't exist.
- Java's compiler refuses to let `StarPlayer` / `RegularPlayer` exist as concrete classes unless they each supply a matching override. If either subclass forgot to implement it, you'd get a **compile error**, not a runtime surprise.
- Your subclasses also use `@Override` — this is a compiler-checked annotation. If the method signature in `StarPlayer` didn't actually match `Player`'s abstract declaration (wrong return type, wrong params), `@Override` forces a compile error rather than silently creating an unrelated overloaded method that dispatch could miss.

The only ways this guarantee could theoretically break (none apply to you):

- If `calculateMonthlySalary()` were `static` — static methods don't participate in dynamic dispatch at all, they resolve by declared type. Yours isn't static.
- If the subclass method had a different signature (different parameter list) — that would be *overloading*, not *overriding*, and would just add a separate method rather than overriding the existing one. `@Override` prevents this from compiling silently.
- If it were `private` — private methods aren't inherited/overridden, just shadowed. Yours is `public`.

None of those apply here, so yes: it's 100% guaranteed `StarPlayer`'s version runs for a `StarPlayer` object, regardless of the variable's declared type. This is genuinely a solid "Critical Thinking" entry for your audit log if you want — it's a place where someone *could* assume Java "might pick the base class version by default," and that assumption would be wrong.